CS 787 - Quiz I Solutions

August 20, 2026

Question 1: Server-Side Stub (Skeleton)

The server-side stub (often called a skeleton) bridges the network and the application logic. The three primary reasons for generating it automatically are:

- **Unmarshalling:** Decoding incoming network byte streams into native types (e.g., extracting arguments).
- **Dispatching/Delegation:** Invoking the actual implementation (servant) method corresponding to the client's request.
- **Marshalling:** Encoding the return value or exceptions back into a network byte stream to send to the client.

```
// Auto-generated Server Stub (Skeleton) Pseudo-code
class MyServiceSkeleton {
private:
    MyServiceImpl* servant; // The actual implementation
public:
    MyServiceSkeleton(MyServiceImpl* impl) : servant(impl) {}

    // Method triggered by the network listener
    void invoke(NetworkStream& stream) {
        int method_id = stream.readInt();

        if (method_id == 1) { // e.g., mapping to a specific
           function
            // 1. Unmarshal arguments
            int arg1 = stream.readInt();

            // 2. Dispatch to actual implementation
            int result = servant->processData(arg1);

            // 3. Marshal return value
            stream.writeInt(result);
            stream.send();
        }
    }
};
```

Question 2: Push-Push Event Channel

In a push-push model, the supplier actively sends (pushes) data to the event channel, and the event channel actively pushes that data to the consumer.

```cpp
#include <iostream>
#include <vector>
#include <string>

using namespace std;

// Interfaces
class PushConsumer {
public:
    virtual void push(const string& eventData) = 0;
    virtual ~PushConsumer() = default;
};

class PushSupplier {
public:
    virtual void disconnect() = 0;
    virtual ~PushSupplier() = default;
};

// Event Channel acting as both Consumer (to Supplier) and
   Supplier (to Consumer)
class EventChannel : public PushConsumer {
private:
    std::vector<PushConsumer*> consumers;
public:
    void registerConsumer(PushConsumer* c) {
        consumers.push_back(c);
    }

    // Pushed by the original supplier
    void push(const string& eventData) override {
        // Push to all registered consumers
        for(auto* consumer : consumers) {
            consumer->push(eventData);
        }
    }
};
```

Question 3: Proxy for Content Caching

The Proxy pattern introduces an intermediate class with the same interface as the real file handler, allowing you to intercept reads and writes to manage an in-memory cache.

```cpp
#include <iostream>
#include <string>
```

```cpp

using namespace std;

class FileInterface {
public:
    virtual string read() = 0;
    virtual void write(const string& data) = 0;
    virtual ~FileInterface() = default;
};

class RealFile : public FileInterface {
public:
    string read() override { return "data_from_disk"; }
    void write(const string& data) override {
        // write to disk implementation
    }
};

class ProxyCacheFile : public FileInterface {
private:
    RealFile* realFile;
    string cache;
    bool isCacheValid;
public:
    ProxyCacheFile(RealFile* rf) : realFile(rf), isCacheValid(
       false) {}

    string read() override {
        if (!isCacheValid) {
            cache = realFile->read(); // fetch and cache
            isCacheValid = true;
        }
        return cache;
    }

    void write(const string& data) override {
        cache = data;
        isCacheValid = true;
        realFile->write(data); // write-through to disk
    }
};
```

Question 4: Polymorphism in Multiple Inheritance

When a class inherits from multiple bases, base class pointers can polymorphically call the overridden methods in the derived class, allowing the object to act as either of its base types.

```cpp
#include <iostream>
```

```cpp

class BaseA {
public:
    virtual void print() { std::cout << "BaseA\n"; }
    virtual ~BaseA() = default;
};

class BaseB {
public:
    virtual void display() { std::cout << "BaseB\n"; }
    virtual ~BaseB() = default;
};

// Multiple Inheritance
class Derived : public BaseA, public BaseB {
public:
    void print() override { std::cout << "Derived printing A\n"; }
    void display() override { std::cout << "Derived displaying B\n"; }
};

int main() {
    Derived d;

    BaseA* ptrA = &d;
    BaseB* ptrB = &d;

    // Polymorphism at work
    ptrA->print();   // Outputs: Derived printing A
    ptrB->display(); // Outputs: Derived displaying B

    return 0;
}
```